

DECISIONES DE SELECCIÓN DE TECNOLOGÍAS

Framework de Evaluación

Cada tecnología la evaluamos con siete criterios ponderados. Madurez y performance empatan con 20% cada uno porque necesitamos tecnología probada que funcione rápido. El ecosistema y la curva de aprendizaje tienen 15% cada uno porque queremos tecnología con buenas librerías y que el equipo pueda aprender fácilmente. Costo, seguridad y escalabilidad tienen 10% cada uno.

Backend con FastAPI

Para el backend de microservicios elegimos FastAPI sobre Flask, Django REST Framework y Express de Node.js. FastAPI maneja 15 mil peticiones por segundo versus 3800 de Flask y 2400 de Django REST. La latencia del percentil 95 es de 42 milisegundos comparado con 180 de Flask y 310 de Django.

Lo que realmente hace brillar a FastAPI es el type safety con Pydantic. Cuando defines que un endpoint recibe un email, Pydantic valida automáticamente que sea un email válido. Si defines que una cédula debe tener 10 dígitos, Pydantic rechaza automáticamente cualquier cosa que no cumpla. Esto elimina toda una categoría de bugs.

FastAPI tiene async y await nativo. Cuando un servicio hace una query a la base de datos que tarda 100 milisegundos, el event loop puede procesar otras peticiones mientras espera. Flask requiere extensiones adicionales para esto y Django recién lo agregó parcialmente. El soporte async es crítico porque nuestros servicios pasan mucho tiempo esperando I/O de bases de datos, Azure Storage y Service Bus.

Dependency injection está incluido en FastAPI. Puedes definir que un endpoint necesita un usuario autenticado, una conexión de base de datos y un cliente de Redis, y FastAPI automáticamente los inyecta. El mismo código de autenticación se reutiliza en todos los endpoints sin duplicación.

FastAPI genera documentación OpenAPI automáticamente. Navegas a /docs y tienes una interfaz Swagger completa donde puedes probar todos los endpoints, ver los schemas de request y response, y entender la API sin leer código. Para /redoc obtienes documentación estilo libro. Esto ahorra semanas de escribir documentación manualmente.

Frontend con Next.js

Next.js 14 ganó sobre Remix, Vite más React puro, y Angular. La característica definitoria es Server Components. Componentes que solo leen datos se ejecutan en el servidor y envían HTML al cliente, no JavaScript.

Streaming SSR significa que Next.js puede enviar el HTML de la página en pedazos conforme está listo. El usuario ve el header y navegación inmediatamente mientras el servidor todavía está cargando la lista de documentos. Esto hace que el Time to First Byte sea menor a 200 milisegundos.

Next.js optimiza imágenes automáticamente. Subes un PNG de 5 megabytes y Next.js lo convierte a WebP o AVIF dependiendo del navegador, lo redimensiona al tamaño exacto que se muestra, y lo sirve desde CDN.

Las API routes de Next.js permiten tener backend y frontend en el mismo proyecto. NextAuth.js maneja toda la autenticación con nuestros microservicios de FastAPI usando estas API routes. No necesitamos configuración compleja de CORS ni gestión de sesiones separada.

PostgreSQL como Base de Datos

PostgreSQL venció a MySQL, MongoDB y SQL Server. PostgreSQL tiene soporte JSONB nativo que combina flexibilidad de schema con performance de SQL. Almacenamos metadatos de documentos en columnas JSONB permitiendo que diferentes tipos de documentos tengan campos personalizados sin migrations constantes de schema.

PostgreSQL mejoró el partitioning declarativo. Las tablas de auditoría que crecen muchísimo se dividen automáticamente en particiones mensuales. Las queries solo buscan en la partición relevante haciendo búsquedas en billones de registros viables.

Row Level Security es genial para multi-tenancy. Defines una política que dice que cada ciudadano solo puede ver sus propios documentos y PostgreSQL lo enforce a nivel de base de datos. Aunque SQL injection burlara la aplicación, el atacante no podría ver datos de otros usuarios porque la base de datos lo bloquea.

Redis para Cache

Redis maneja cache, rate limiting, circuit breaker state y distributed locks. La latencia promedio es menor a 1 milisegundos. Para rate limiting, incrementamos un contador en Redis por cada petición y si pasa el límite del tier del usuario, rechazamos la petición. El contador expira automáticamente después de un minuto.

Los distributed locks usando RedLock pattern aseguran que solo un worker procese cada documento. Si dos workers agarran el mismo mensaje de la cola por una race condition, solo uno adquiere el lock de Redis y procesa el documento. El otro lo suelta de vuelta a la cola.

Redis persiste datos con snapshots RDB cada 15 minutos y append-only file para cada escritura. Si Redis se cae, cuando reinicia recupera el estado de los últimos 15 minutos de snapshot más todos los cambios desde entonces del AOF.

Pub/Sub de Redis permite notificaciones en tiempo real. Cuando un documento cambia de estado, publicamos un evento que todos los clientes conectados reciben instantáneamente para actualizar la UI sin polling.

Kubernetes para Orquestación

Kubernetes ganó sobre Docker Swarm, Nomad y AWS ECS/Fargate. La complejidad de Kubernetes es alta pero el ecosistema compensa ampliamente. El Horizontal Pod Autoscaler escala servicios basándose en CPU y memoria. Cuando el uso de CPU pasa de 70%, Kubernetes automáticamente crea más réplicas hasta el máximo configurado de 10 pods.

External Secrets Operator sincroniza secretos de Key Vault a Kubernetes cada 5 minutos automáticamente. Cuando rotas un secreto en Key Vault, los pods lo detectan y se recargan sin downtime.

Network Policies implementan micro-segmentación. El servicio de ingestion solo puede hablar con Service Bus y PostgreSQL. El servicio de signature solo puede hablar con Blob Storage y Key Vault. Si un atacante compromete un servicio, no puede moverse lateralmente a otros servicios.

Pod Disruption Budgets aseguran que siempre haya mínimo 2 réplicas de cada servicio crítico corriendo. Durante actualizaciones o cuando un nodo falla, Kubernetes no destruye pods hasta que haya suficientes réplicas saludables en otros nodos.

Autenticación con JWT y bcrypt

Decidimos implementar autenticación propia usando JWT tokens en lugar de servicios externos como Auth0 o Azure AD B2C. Esto nos da control total sobre el flujo de autenticación y reduce dependencias externas que podrían fallar o cambiar precios.

Usamos bcrypt para hashear contraseñas con un work factor de 12 rondas. Bcrypt es resistente a ataques de fuerza bruta porque cada hash toma aproximadamente 250 milisegundos en calcular, haciendo imposible probar millones de contraseñas por segundo. Cuando un usuario cambia su contraseña, automáticamente rehashamos con el work factor más reciente.

Los JWT tokens se firman con algoritmo RS256 usando una clave privada RSA-4096 almacenada en Azure Key Vault. La clave pública está disponible en un endpoint /auth/jwks.json que todos los servicios consultan para validar tokens. Los access tokens expiran en 15 minutos forzando reautenticación frecuente. Los refresh tokens duran 7 días y se almacenan en PostgreSQL con la capacidad de revocarlos individualmente.

El servicio de autenticación implementa rate limiting estricto. Solo permite 5 intentos de login por IP por minuto. Después del quinto intento fallido, la IP se bloquea por 15 minutos en Redis. Si una cuenta específica recibe 10 intentos fallidos en una hora, se bloquea completamente y requiere reset de contraseña por email.

Service Bus para Mensajería

Azure Service Bus ganó sobre RabbitMQ y Apache Kafka. Service Bus es completamente administrado eliminando la carga operativa de mantener un cluster de mensajería.

Service Bus soporta sessions que garantizan que todos los mensajes de un ciudadano específico se procesen en orden y por el mismo worker. Esto es crítico para mantener consistencia cuando un usuario sube múltiples documentos rápidamente.

La integración nativa con Kubernetes permite que cuando la longitud de una cola crece, el Horizontal Pod Autoscaler crea más workers automáticamente. Cuando la cola está vacía, escala a cero ahorrando costos de computación.

Conclusión de Tecnologías

El stack completo es FastAPI en el backend por performance y type safety, Next.js en el frontend por Server Components y optimizaciones automáticas, PostgreSQL como base de datos relacional por JSONB y Row Level Security, Redis para cache y distributed locks, Azure Service Bus para mensajería asíncrona con dead letter queues, Kubernetes en AKS para orquestación con External Secrets Operator, JWT con RS256 para autenticación stateless, bcrypt para hashing de contraseñas.

Este stack permite escalar rápidamente la aplicación sin cambios arquitectónicos fundamentales. Cumple todas las regulaciones gubernamentales de seguridad y archivo. La combinación de tecnologías probadas en producción nos da confianza para operar un sistema crítico gubernamental 24/7.